

Truy xuất và Lưu trữ Dữ liệu với Python

Nhập, tích hợp và lưu trữ dữ liệu phục vụ phân tích

DSAI1005 - Phân tích dữ liệu với Python

Khoa Khoa học Dữ liệu và Trí tuệ Nhân tạo
Đại học Kinh tế Quốc dân

2026

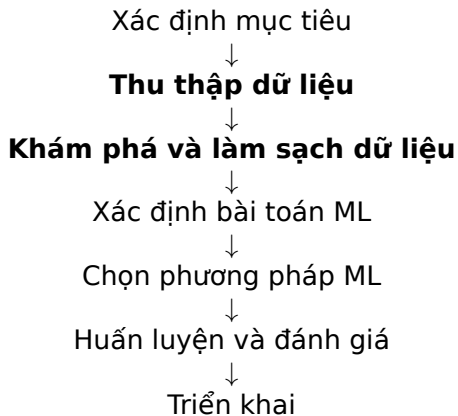
Động lực của bài học

Trong các dự án Data Science và Machine Learning, dữ liệu hiếm khi đã có sẵn dưới dạng một DataFrame sạch và sẵn sàng cho phân tích.

Dữ liệu có thể được lưu ở nhiều định dạng:

- CSV và các tệp văn bản có dấu phân cách;
- workbook Excel;
- JSON trả về từ Web API;
- bảng HTML trên website;
- báo cáo và tài liệu PDF;
- cơ sở dữ liệu quan hệ và cơ sở dữ liệu NoSQL.

Vị trí trong quy trình Machine Learning



Bài học tập trung vào các công việc cần thực hiện trước khi xây dựng mô hình:

- truy xuất dữ liệu;
- chuẩn bị dữ liệu;
- tích hợp nhiều nguồn;
- lưu kết quả để tái sử dụng.

Mục tiêu học tập

Sau bài học này, người học có thể:

- truy xuất dữ liệu từ CSV, Excel, JSON, HTML, PDF, SQLite và MongoDB;
- sử dụng các tham số I/O quan trọng của Pandas như `usecols`, `dtype`, `parse_dates` và `chunksize`;
- chuyển JSON lồng nhau về dạng bảng;
- chuyển dữ liệu giữa SQL database và Pandas DataFrame;
- xây dựng pipeline dữ liệu đa nguồn;
- lựa chọn định dạng lưu trữ phù hợp cho từng nhiệm vụ phân tích.

Cấu trúc bài học

- 1 Tổng quan về truy xuất và lưu trữ dữ liệu
- 2 CSV với NumPy
- 3 CSV với Pandas
- 4 Microsoft Excel
- 5 JSON
- 6 HTML và PDF
- 7 SQLite
- 8 MongoDB và PyMongo
- 9 Pipeline dữ liệu đa nguồn
- 10 Các nguyên tắc thực hành tốt

Điều kiện tiên quyết

Người học nên có:

- kiến thức Python cơ bản;
- hiểu biến, list, dictionary, vòng lặp và hàm;
- kiến thức cơ bản về NumPy và Pandas DataFrame;
- hiểu khái niệm hàng, cột và dữ liệu dạng bảng;
- môi trường Jupyter Notebook, Google Colab, VS Code hoặc tương đương.

Lộ trình

- 1 Tổng quan về truy xuất và lưu trữ dữ liệu
- 2 Làm việc với CSV bằng NumPy
- 3 Làm việc với CSV bằng Pandas
- 4 Làm việc với Microsoft Excel
- 5 Làm việc với JSON
- 6 Truy xuất dữ liệu từ HTML và PDF
- 7 Làm việc với SQLite
- 8 Làm việc với MongoDB và PyMongo
- 9 Xây dựng pipeline dữ liệu đa nguồn
- 10 Các nguyên tắc thực hành tốt

Truy xuất dữ liệu là gì?

Truy xuất dữ liệu là quá trình lấy dữ liệu từ một nguồn lưu trữ và đưa vào môi trường phân tích.

Ví dụ: đọc tệp CSV vào Pandas.

```
customers.csv  
  |  
  v  
pd.read_csv()  
  |  
  v  
DataFrame
```

Ví dụ: truy vấn database trước khi phân tích.

```
SQL Database -> SQL Query -> Pandas DataFrame
```


Lưu trữ dữ liệu là gì?

Lưu trữ dữ liệu là quá trình ghi dữ liệu hoặc kết quả phân tích vào một định dạng hay hệ thống có thể tái sử dụng.

```
DataFrame -> to_csv() -> report.csv  
DataFrame -> to_sql() -> Database Table
```

Lưu trữ không chỉ là bước xuất kết quả. Nó còn hỗ trợ tái lập phân tích, báo cáo, chia sẻ dữ liệu và xây dựng mô hình sau này.

Các nguồn dữ liệu phổ biến

Nhóm nguồn dữ liệu	Ví dụ
Tệp phẳng	CSV, TSV
Bảng tính	Excel
Dữ liệu bán cấu trúc	JSON
Dữ liệu Web	bảng HTML
Tài liệu	báo cáo PDF
Cơ sở dữ liệu quan hệ	SQLite, PostgreSQL, MySQL
Cơ sở dữ liệu NoSQL	MongoDB

Dữ liệu có cấu trúc và bán cấu trúc

Dữ liệu có cấu trúc thường có dạng hàng-cột rõ ràng.

CustomerID	Name	City	Revenue
------------	------	------	---------

Dữ liệu bán cấu trúc có thể chứa đối tượng và danh sách lồng nhau.

```
{
  "customer": {"name": "An", "city": "Hanoi"},
  "orders": [
    {"product": "A"},
    {"product": "B"}
  ]
}
```

Bài tập: Phân loại nguồn dữ liệu

Phân loại các nguồn sau vào nhóm phù hợp.

- 1 sales.csv
- 2 customers.xlsx
- 3 Dữ liệu JSON trả về từ Web API
- 4 company.db
- 5 MongoDB
- 6 Báo cáo tài chính dạng PDF

Các nhóm cần dùng: tệp phẳng, bảng tính, dữ liệu bán cấu trúc, cơ sở dữ liệu quan hệ, cơ sở dữ liệu NoSQL, tài liệu.

Kiểm tra kiến thức

Câu 1. Định dạng nào thường được xem là tệp phẳng?

- A. CSV
- B. MongoDB
- C. SQLite
- D. PDF

Câu 2. Định dạng nào hỗ trợ tự nhiên đối tượng và danh sách lồng nhau?

- A. JSON
- B. CSV
- C. TSV
- D. Ma trận số

Lộ trình

- 1 Tổng quan về truy xuất và lưu trữ dữ liệu
- 2 Làm việc với CSV bằng NumPy**
- 3 Làm việc với CSV bằng Pandas
- 4 Làm việc với Microsoft Excel
- 5 Làm việc với JSON
- 6 Truy xuất dữ liệu từ HTML và PDF
- 7 Làm việc với SQLite
- 8 Làm việc với MongoDB và PyMongo
- 9 Xây dựng pipeline dữ liệu đa nguồn
- 10 Các nguyên tắc thực hành tốt

Đọc dữ liệu số với `np.loadtxt()`

`np.loadtxt()` phù hợp khi dữ liệu chủ yếu là số, có cấu trúc đơn giản và không có giá trị thiếu.

```
import numpy as np

data = np.loadtxt(
    "matrix_data.csv",
    delimiter=";",
    skiprows=1
)

print(data)
print(data.shape)
```

Có thể tính trung bình từng cột bằng:

```
print(np.mean(data, axis=0))
```

Bài tập: `np.loadtxt()`

Hoàn thiện chương trình để đọc `sales.csv`, trong đó các giá trị được phân cách bằng dấu phẩy.

```
# data = np.loadtxt(  
# "sales.csv",  
# delimiter=...  
# )  
  
# print(data.shape)
```

Sau đó xác định số hàng và số cột của dữ liệu.

Xử lý giá trị thiếu với `np.genfromtxt()`

`np.genfromtxt()` linh hoạt hơn `np.loadtxt()` khi tệp số chứa giá trị thiếu.

```
data = np.genfromtxt(  
    "numeric_data.csv",  
    delimiter=",",  
    skip_header=1,  
    filling_values=0.0,  
    dtype=float  
)  
  
print(data)
```

Trong ví dụ này, các giá trị thiếu được thay bằng 0.0.

Ghi mảng với `np.savetxt()`

`np.savetxt()` dùng để lưu mảng NumPy ra tệp văn bản.

```
output = np.random.randn(5, 3)
```

```
np.savetxt(  
    "output.csv",  
    output,  
    delimiter="," ,  
    fmt="%.2f",  
    header="A,B,C",  
    comments=""  
)
```

`fmt="%.2f"` ghi mỗi số với hai chữ số sau dấu thập phân.

Các hàm CSV chính trong NumPy

Hàm	Công dụng
<code>np.loadtxt()</code>	Đọc dữ liệu số từ tệp văn bản có cấu trúc đơn giản
<code>np.genfromtxt()</code>	Đọc dữ liệu số và hỗ trợ giá trị thiếu
<code>np.savetxt()</code>	Ghi mảng NumPy ra tệp văn bản

Bài thực hành: Monthly Sales

Tạo `monthly_sales.csv` với nội dung:

```
Month,Revenue,Cost  
1,120,80  
2,150,  
3,180,110  
4,,130
```

Thực hiện:

- 1 Đọc dữ liệu bằng `np.genfromtxt()`.
- 2 Thay giá trị thiếu bằng 0.
- 3 Tính trung bình Revenue và Cost.
- 4 Lưu dữ liệu sau xử lý bằng `np.savetxt()`.

Lộ trình

- 1 Tổng quan về truy xuất và lưu trữ dữ liệu
- 2 Làm việc với CSV bằng NumPy
- 3 Làm việc với CSV bằng Pandas**
- 4 Làm việc với Microsoft Excel
- 5 Làm việc với JSON
- 6 Truy xuất dữ liệu từ HTML và PDF
- 7 Làm việc với SQLite
- 8 Làm việc với MongoDB và PyMongo
- 9 Xây dựng pipeline dữ liệu đa nguồn
- 10 Các nguyên tắc thực hành tốt

Đọc CSV với `pd.read_csv()`

Pandas cung cấp `pd.read_csv()` để đọc dữ liệu dạng bảng từ tệp CSV.

```
import pandas as pd  
  
df = pd.read_csv("customers.csv")  
  
print(df.head())
```

Pandas phù hợp khi dữ liệu chứa văn bản, ngày tháng, nhiều kiểu dữ liệu, giá trị thiếu và tên cột.

Xác định dấu phân cách với sep

Không phải mọi tệp đều dùng dấu phẩy làm ký tự phân cách.
Tệp phân cách bằng dấu chấm phẩy:

```
df = pd.read_csv(  
    "data.csv",  
    sep=";"  
)
```

Tệp TSV:

```
df = pd.read_csv(  
    "data.tsv",  
    sep="\t"  
)
```

Chỉ đọc cột cần thiết với usecols

usecols cho phép chọn các cột cần đọc ngay khi nhập dữ liệu.

```
df = pd.read_csv(  
    "customers.csv",  
    usecols=[  
        "customer_id",  
        "city",  
        "total_spent"  
    ]  
)
```

Cách này giúp giảm RAM, rút ngắn thời gian đọc và loại bỏ sớm các biến không cần thiết.

Quy định kiểu dữ liệu với dtype

Pandas có thể tự suy luận kiểu dữ liệu, nhưng nhiều trường hợp cần quy định tường minh.

```
df = pd.read_csv(  
    "customers.csv",  
    dtype={  
        "customer_id": str,  
        "age": "Int64"  
    }  
)
```

Các trường ID thường là mã định danh, không phải đại lượng số học.

Chuyển ngày tháng với parse_dates

Có thể yêu cầu Pandas chuyển cột ngày tháng ngay khi đọc dữ liệu.

```
df = pd.read_csv(  
    "orders.csv",  
    parse_dates=["OrderDate"]  
)  
  
print(df.dtypes)
```

Cách này hữu ích khi cần lọc, sắp xếp hoặc tính toán theo thời gian.

Xác định giá trị thiếu với `na_values`

Trong dữ liệu thực tế, giá trị thiếu có thể được biểu diễn bằng nhiều ký hiệu khác nhau.

```
df = pd.read_csv(  
    "customers.csv",  
    na_values=[  
        "NA",  
        "N/A",  
        "-",  
        "missing"  
    ]  
)
```

Pandas sẽ chuyển các ký hiệu này thành giá trị thiếu chuẩn.

Mã hóa ký tự

Khi làm việc với dữ liệu tiếng Việt, UTF-8 là lựa chọn phổ biến.

```
df = pd.read_csv(  
    "customers.csv",  
    encoding="utf-8"  
)
```

Khi xuất CSV để mở bằng Excel trên Windows:

```
df.to_csv(  
    "customers_clean.csv",  
    index=False,  
    encoding="utf-8-sig"  
)
```

Kiểm tra dữ liệu sau khi nhập

Sau khi đọc dữ liệu, cần kiểm tra ngay cấu trúc và chất lượng của DataFrame.

```
df.head()  
df.shape  
df.info()  
df.dtypes  
df.isna().sum()
```

Các lệnh này giúp kiểm tra số hàng, số cột, tên cột, kiểu dữ liệu và số giá trị thiếu.

Đọc dữ liệu lớn với chunksize

Khi tệp CSV lớn hơn bộ nhớ khả dụng, không nên đọc toàn bộ dữ liệu cùng lúc.

```
total = 0

for chunk in pd.read_csv(
    "transactions.csv",
    chunksize=100000
):
    total += chunk["Amount"].sum()

print(total)
```

chunksize cho phép xử lý dữ liệu tuần tự theo từng khối.

Ghi DataFrame ra CSV

Sử dụng phương thức `to_csv()`.

```
df.to_csv(  
    "clean_data.csv",  
    index=False,  
    encoding="utf-8-sig"  
)
```

`index=False` giúp không ghi chỉ mục của DataFrame thành một cột bổ sung.

Các hàm và tham số CSV chính của Pandas

Hàm hoặc tham số	Công dụng
------------------	-----------

<code>pd.read_csv()</code>	Đọc CSV hoặc tệp văn bản có dấu phân cách
<code>usecols=</code>	Chỉ đọc các cột cần thiết
<code>dtype=</code>	Quy định kiểu dữ liệu
<code>parse_dates=</code>	Chuyển cột sang kiểu ngày tháng
<code>na_values=</code>	Xác định ký hiệu giá trị thiếu
<code>chunksize=</code>	Đọc dữ liệu theo từng khối
<code>df.to_csv()</code>	Ghi DataFrame ra CSV

Kiểm tra kiến thức: Pandas CSV

Câu 1. Tham số nào cho phép chỉ đọc một số cột được lựa chọn?

- A. usecols
- B. keepcols
- C. columns_only
- D. filter_columns

Câu 2. Tham số nào hữu ích khi xử lý tệp CSV rất lớn?

- A. chunksize
- B. batch=True
- C. split=True
- D. large=True

Bài thực hành: Customer CSV

Tạo `customers.csv` gồm các cột: `CustomerID`, `Name`, `City`, `Age`, `SignupDate` và `TotalSpent`.

Trong dữ liệu, đưa vào một số giá trị `NA`, một số giá trị `-` và `CustomerID` dạng `001`, `002`, ...

Thực hiện:

- 1 Đọc `CustomerID` dưới dạng `str`.
- 2 Chuyển `SignupDate` sang `datetime`.
- 3 Xem `NA` và `-` là giá trị thiếu.
- 4 Kiểm tra bằng `head()`, `shape`, `info()` và `isna().sum()`.
- 5 Xuất kết quả sang `customers_clean.csv`.

Lộ trình

- 1 Tổng quan về truy xuất và lưu trữ dữ liệu
- 2 Làm việc với CSV bằng NumPy
- 3 Làm việc với CSV bằng Pandas
- 4 Làm việc với Microsoft Excel**
- 5 Làm việc với JSON
- 6 Truy xuất dữ liệu từ HTML và PDF
- 7 Làm việc với SQLite
- 8 Làm việc với MongoDB và PyMongo
- 9 Xây dựng pipeline dữ liệu đa nguồn
- 10 Các nguyên tắc thực hành tốt

Đọc một trang tính

Pandas sử dụng `pd.read_excel()` để đọc dữ liệu Excel.

```
df = pd.read_excel(  
    "products.xlsx",  
    sheet_name="Products",  
    engine="openpyxl"  
)
```

`sheet_name` xác định trang tính cần đọc.

Đọc toàn bộ trang tính

Có thể đọc toàn bộ workbook bằng `sheet_name=None`.

```
workbook = pd.read_excel(  
    "products.xlsx",  
    sheet_name=None  
)  
  
print(workbook.keys())
```

Kết quả là dictionary: key là tên trang tính, value là DataFrame tương ứng.

Ghi nhiều DataFrame vào một workbook

`pd.ExcelWriter()` cho phép ghi nhiều DataFrame vào các trang tính khác nhau trong cùng một tệp Excel.

```
with pd.ExcelWriter(
    "report.xlsx",
    engine="openpyxl"
) as writer:
    df_products.to_excel(
        writer,
        sheet_name="Products",
        index=False
    )
    df_sales.to_excel(
        writer,
        sheet_name="Sales",
        index=False
    )
```

Các hàm Excel chính

Hàm hoặc tham số	Công dụng
<code>pd.read_excel()</code>	Đọc dữ liệu Excel
<code>sheet_name=</code>	Chọn trang tính
<code>sheet_name=None</code>	Đọc toàn bộ trang tính
<code>df.to_excel()</code>	Ghi DataFrame vào Excel
<code>pd.ExcelWriter()</code>	Ghi nhiều DataFrame vào một workbook

Bài tập và kiểm tra kiến thức về Excel

Bài tập. Tạo `business_data.xlsx` gồm ba trang tính: Customers, Products và Sales. Sau đó đọc toàn bộ workbook, in tên các trang tính, tính tổng Sales và xuất workbook mới.

Câu hỏi. `sheet_name=None` trả về đối tượng nào?

- A. Dictionary chứa các DataFrame
- B. Chỉ trang tính đầu tiên
- C. Một mảng NumPy
- D. Một chuỗi ký tự

Lộ trình

- 1 Tổng quan về truy xuất và lưu trữ dữ liệu
- 2 Làm việc với CSV bằng NumPy
- 3 Làm việc với CSV bằng Pandas
- 4 Làm việc với Microsoft Excel
- 5 Làm việc với JSON**
- 6 Truy xuất dữ liệu từ HTML và PDF
- 7 Làm việc với SQLite
- 8 Làm việc với MongoDB và PyMongo
- 9 Xây dựng pipeline dữ liệu đa nguồn
- 10 Các nguyên tắc thực hành tốt

JSON là gì?

JSON là định dạng dữ liệu phổ biến trong Web API, thương mại điện tử, microservices và cơ sở dữ liệu NoSQL.

```
[  
  {"customer_id": "C001", "city": "Hanoi"},  
  {"customer_id": "C002", "city": "Danang"}  
]
```

Với JSON tương đối phẳng, có thể dùng:

```
df = pd.read_json("customers.json")
```

JSON lồng nhau

Dữ liệu JSON thực tế thường có nhiều cấp.

```
{  
  "order_id": "0001",  
  "customer": {  
    "customer_id": "C001",  
    "city": "Hanoi"  
  },  
  "items": [  
    {"sku": "P001", "qty": 2},  
    {"sku": "P002", "qty": 1}  
  ]  
}
```

Để phân tích bằng Pandas, thường cần chuyển cấu trúc này thành bảng.

Làm phẳng với `pd.json_normalize()`

```
df_items = pd.json_normalize(  
    orders,  
    record_path=["items"],  
    meta=[  
        "order_id",  
        ["customer", "customer_id"],  
        ["customer", "city"]  
    ]  
)
```

- `record_path`: danh sách lồng nhau cần mở rộng thành các dòng.
- `meta`: các thuộc tính ở cấp cha cần giữ lại.

Bài tập và kiểm tra kiến thức về JSON

Bài tập. Tạo dữ liệu gồm ba đơn hàng. Mỗi đơn hàng có Customer và Items. Dùng `pd.json_normalize()` để mỗi item trở thành một dòng.

Câu hỏi. Vai trò của `record_path` là gì?

- A. Xác định danh sách lồng nhau cần mở rộng thành các dòng
- B. Đổi tên cột
- C. Ghi dữ liệu ra JSON
- D. Xóa giá trị thiếu

Lộ trình

- 1 Tổng quan về truy xuất và lưu trữ dữ liệu
- 2 Làm việc với CSV bằng NumPy
- 3 Làm việc với CSV bằng Pandas
- 4 Làm việc với Microsoft Excel
- 5 Làm việc với JSON
- 6 Truy xuất dữ liệu từ HTML và PDF**
- 7 Làm việc với SQLite
- 8 Làm việc với MongoDB và PyMongo
- 9 Xây dựng pipeline dữ liệu đa nguồn
- 10 Các nguyên tắc thực hành tốt

Đọc bảng HTML

`pd.read_html()` có thể phát hiện các phần tử `<table>` và chuyển chúng thành `DataFrame`.

```
tables = pd.read_html(  
    "financial_quotes.html"  
)  
  
print(len(tables))  
  
stocks = tables[0]
```

Kết quả là danh sách các `DataFrame`.

Vì sao PDF khó trích xuất hơn CSV?

CSV được thiết kế để biểu diễn dữ liệu dạng bảng.

Ngược lại, PDF chủ yếu được thiết kế để:

- hiển thị nội dung;
- bảo toàn bố cục;
- phục vụ in ấn.

Vì vậy, cấu trúc hàng-cột nhìn thấy trên PDF không phải lúc nào cũng được lưu dưới dạng bảng tường minh bên trong tệp.

Trích xuất bảng với pdfplumber

Một thư viện có thể dùng để trích xuất bảng từ PDF là pdfplumber.

```
import pdfplumber

with pdfplumber.open("report.pdf") as pdf:
    page = pdf.pages[0]
    tables = page.extract_tables()
```

Chuyển bảng sang DataFrame:

```
table = tables[0]

df = pd.DataFrame(
    table[1:],
    columns=table[0]
)
```

Bài tập HTML và PDF

HTML. Cho một tệp HTML có bảng giá sản phẩm và bảng tỷ giá. Đọc cả hai bảng, kiểm tra cấu trúc và lưu mỗi bảng vào một trang tính Excel.

PDF. Với một bảng trích xuất từ PDF, hãy kiểm tra:

- tiêu đề cột;
- số hàng và số cột;
- giá trị thiếu;
- các bước làm sạch cần thực hiện trước khi phân tích.

Lộ trình

- 1 Tổng quan về truy xuất và lưu trữ dữ liệu
- 2 Làm việc với CSV bằng NumPy
- 3 Làm việc với CSV bằng Pandas
- 4 Làm việc với Microsoft Excel
- 5 Làm việc với JSON
- 6 Truy xuất dữ liệu từ HTML và PDF
- 7 Làm việc với SQLite**
- 8 Làm việc với MongoDB và PyMongo
- 9 Xây dựng pipeline dữ liệu đa nguồn
- 10 Các nguyên tắc thực hành tốt

SQLite là gì?

SQLite là hệ quản trị cơ sở dữ liệu quan hệ gọn nhẹ. Toàn bộ cơ sở dữ liệu có thể lưu trong một tệp.

```
company.db
```

Python hỗ trợ SQLite thông qua module chuẩn `sqlite3`.

```
import sqlite3

conn = sqlite3.connect("company.db")
cursor = conn.cursor()
```

Thực hiện truy vấn SQL

```
cursor.execute(
    """
    SELECT emp_id, full_name, salary
    FROM employees
    WHERE salary >= ?
    """,
    (300000000,)
)

rows = cursor.fetchall()
```

`fetchall()` trả về toàn bộ các dòng thỏa mãn truy vấn.

Truy vấn có tham số

Khi giá trị truy vấn đến từ biến Python hoặc dữ liệu người dùng, nên sử dụng truy vấn có tham số.

```
cursor.execute(  
    "SELECT * FROM users WHERE id = ?",  
    (user_id,) )
```

Cách này tách cấu trúc SQL khỏi giá trị được truyền vào và giúp giảm rủi ro SQL Injection.

Đọc kết quả SQL vào Pandas

Pandas có thể đọc trực tiếp kết quả của một truy vấn SQL.

```
query = """
SELECT
    emp_id,
    full_name,
    salary
FROM employees
"""
```

```
df = pd.read_sql_query(
    query,
    conn
)
```

Kết quả được trả về dưới dạng DataFrame.

Kết hợp nhiều bảng bằng JOIN

```
query = """
SELECT
    e.emp_id,
    e.full_name,
    d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
"""
```

```
df = pd.read_sql_query(query, conn)
```


Ghi DataFrame vào SQLite

Pandas cung cấp phương thức `to_sql()`.

```
df.to_sql(  
    name="sales_summary",  
    con=conn,  
    if_exists="replace",  
    index=False  
)
```

Các lựa chọn thường dùng của `if_exists`: `fail`, `replace`, `append`.

Các hàm SQLite và bài tập

Hàm	Công dụng
<code>sqlite3.connect()</code>	Mở kết nối đến SQLite
<code>cursor.execute()</code>	Thực thi câu lệnh SQL
<code>fetchall()</code>	Lấy kết quả truy vấn
<code>pd.read_sql_query()</code>	Chuyển kết quả SQL thành DataFrame
<code>df.to_sql()</code>	Ghi DataFrame vào bảng SQL

Bài tập. Tạo `sales.db`, JOIN `orders` với `customers`, tính tổng `Amount` theo `City` và lưu kết quả vào `city_sales_summary`.

Lộ trình

- 1 Tổng quan về truy xuất và lưu trữ dữ liệu
- 2 Làm việc với CSV bằng NumPy
- 3 Làm việc với CSV bằng Pandas
- 4 Làm việc với Microsoft Excel
- 5 Làm việc với JSON
- 6 Truy xuất dữ liệu từ HTML và PDF
- 7 Làm việc với SQLite
- 8 Làm việc với MongoDB và PyMongo**
- 9 Xây dựng pipeline dữ liệu đa nguồn
- 10 Các nguyên tắc thực hành tốt

MongoDB là gì?

MongoDB là hệ quản trị cơ sở dữ liệu NoSQL hướng document.

Cơ sở dữ liệu quan hệ:

Table
-> Row

MongoDB:

Collection
-> Document

Document có thể chứa giá trị đơn, đối tượng lồng nhau, danh sách và nhiều cấp cấu trúc.

Kết nối MongoDB

PyMongo cung cấp MongoClient để kết nối MongoDB.

```
from pymongo import MongoClient

client = MongoClient(
    "mongodb://localhost:27017/"
)

db = client["retail_db"]
orders = db["orders"]
```

Các thao tác CRUD

CRUD gồm Create, Read, Update và Delete.

```
# Create
orders.insert_one(sample_order)

# Read one document
orders.find_one({"order_id": "ORD001"})

# Filter multiple documents
orders.find({"total_amount": {"$gte": 1000000}})

# Update
orders.update_one(
    {"order_id": "ORD001"},
    {"$set": {"status": "REFUNDED"}}
)
```

Chuyển dữ liệu MongoDB sang Pandas

Kết quả truy vấn có thể được chuyển thành danh sách các document.

```
documents = list(
    orders.find()
)

df = pd.json_normalize(
    documents
)
```

Trường `_id` có thể được chuyển sang chuỗi trước khi xuất dữ liệu.

```
df["_id"] = df["_id"].astype(str)
```

Bài tập và kiểm tra kiến thức về MongoDB

Bài tập. Tạo collection `customer_orders` có ít nhất năm document. Thêm đơn hàng, truy vấn các đơn hàng đã thanh toán, cập nhật trạng thái một đơn hàng, chuyển kết quả sang Pandas và tính doanh thu đã thanh toán.

Câu hỏi. Hàm Pandas nào hữu ích với document MongoDB lồng nhau?

- A. `pd.json_normalize()`
- B. `pd.read_csv()`
- C. `pd.crosstab()`
- D. `pd.cut()`

Lộ trình

- 1 Tổng quan về truy xuất và lưu trữ dữ liệu
- 2 Làm việc với CSV bằng NumPy
- 3 Làm việc với CSV bằng Pandas
- 4 Làm việc với Microsoft Excel
- 5 Làm việc với JSON
- 6 Truy xuất dữ liệu từ HTML và PDF
- 7 Làm việc với SQLite
- 8 Làm việc với MongoDB và PyMongo
- 9 Xây dựng pipeline dữ liệu đa nguồn**
- 10 Các nguyên tắc thực hành tốt

Bối cảnh pipeline

Một doanh nghiệp lưu dữ liệu ở nhiều nguồn:

```
customers.csv  
products.xlsx  
orders.json  
company.db
```

Mục tiêu là tích hợp các nguồn này thành một tập dữ liệu phân tích thống nhất phục vụ báo cáo, trực quan hóa hoặc Machine Learning.

Quy trình tích hợp tổng quát

CSV / Excel / JSON / SQL



Data Ingestion



Data Cleaning



Transformation



Merge



Analytics Dataset



SQLite

Excel Report

Đọc nhiều nguồn dữ liệu

```
customers = pd.read_csv("customers.csv")

products = pd.read_excel(
    "products.xlsx",
    sheet_name="Products"
)

import json
with open("orders.json", "r", encoding="utf-8") as f:
    orders = json.load(f)
```

Làm phẳng đơn hàng và tạo biến

```
items = pd.json_normalize(  
    orders,  
    record_path=["items"],  
    meta=[  
        "order_id",  
        ["customer", "customer_id"]  
    ]  
)  
  
items["Revenue"] = (  
    items["Quantity"] * items["Price"]  
)
```

Tích hợp dữ liệu với merge ()

```
df = items.merge(  
    products,  
    on="ProductID",  
    how="left"  
)  
  
df = df.merge(  
    customers,  
    on="CustomerID",  
    how="left"  
)
```

Sau mỗi lần merge, cần kiểm tra:

```
print(df.shape)  
print(df.isna().sum())
```

Tạo bảng KPI

Tính doanh thu và số đơn hàng theo khu vực.

```
region_summary = (  
    df  
    .groupby("Region")  
    .agg(  
        Revenue=("Revenue", "sum"),  
        Orders=("OrderID", "nunique")  
    )  
    .reset_index()  
)
```

Lưu trữ và báo cáo kết quả

Lưu dữ liệu phân tích vào SQLite.

```
conn = sqlite3.connect("analytics.db")

df.to_sql(
    "order_analytics",
    conn,
    if_exists="replace",
    index=False
)

conn.close()
```

Xuất báo cáo KPI ra Excel.

```
with pd.ExcelWriter("dashboard.xlsx") as writer:
    df.to_excel(writer, sheet_name="Details", index=False)
    region_summary.to_excel(writer, sheet_name="Region_KPI", index=False)
```


Bài tập project: Tích hợp dữ liệu đa nguồn

Xây dựng `order_analytics` từ `customers.csv`, `products.xlsx` và `orders.json`.

Các trường cần có:

- OrderID, CustomerID, CustomerName, City
- ProductID, Category, Quantity, UnitPrice, Revenue

Nhiệm vụ: đọc nguồn, kiểm tra dữ liệu, làm phẳng JSON, merge bảng, xác định bản ghi không khớp, tính revenue, lưu chi tiết vào SQLite và xuất KPI ra Excel.

Lộ trình

- 1 Tổng quan về truy xuất và lưu trữ dữ liệu
- 2 Làm việc với CSV bằng NumPy
- 3 Làm việc với CSV bằng Pandas
- 4 Làm việc với Microsoft Excel
- 5 Làm việc với JSON
- 6 Truy xuất dữ liệu từ HTML và PDF
- 7 Làm việc với SQLite
- 8 Làm việc với MongoDB và PyMongo
- 9 Xây dựng pipeline dữ liệu đa nguồn
- 10 Các nguyên tắc thực hành tốt**

Chỉ đọc các cột cần thiết

Nếu tệp có nhiều cột nhưng chỉ cần một số biến, hãy chọn cột ngay khi đọc.

```
pd.read_csv(  
    "large.csv",  
    usecols=[  
        "CustomerID",  
        "Sales"  
    ]  
)
```

Cách này giúp giảm thời gian đọc và bộ nhớ sử dụng.

Kiểm tra sớm và thường xuyên

Kiểm tra ngay sau khi nhập dữ liệu.

```
df.head()  
df.shape  
df.info()  
df.isna().sum()
```

Kiểm tra lại sau khi tích hợp.

```
df.isna().sum()
```

Giá trị thiếu mới có thể xuất hiện khi khóa giữa các nguồn không khớp.

Kiểm soát kiểu dữ liệu

Kiểm tra kiểu dữ liệu bằng:

```
df.dtypes
```

Không nên giả định mọi trường chứa chữ số đều là đại lượng số học. Ví dụ, CustomerID = "001" là mã định danh và thường nên được lưu dưới dạng chuỗi.

Sử dụng context manager

Khi làm việc với tệp, nên dùng câu lệnh `with`.

```
with open("data.json", encoding="utf-8") as f:  
    data = json.load(f)
```

Khi tạo báo cáo Excel:

```
with pd.ExcelWriter("report.xlsx") as writer:  
    summary.to_excel(writer, index=False)
```

Tài nguyên sẽ được đóng đúng cách sau khi thao tác hoàn tất.

Lựa chọn định dạng lưu trữ phù hợp

Yêu cầu	Định dạng phù hợp
Trao đổi bảng dữ liệu đơn giản	CSV
Tạo báo cáo nhiều trang tính	Excel
Trao đổi dữ liệu qua Web API	JSON
Lưu dữ liệu quan hệ và truy vấn	SQL
Lưu document lồng nhau linh hoạt	MongoDB
Lưu bảng dữ liệu lớn	Parquet
Trích xuất bảng trên website	HTML
Phân phối báo cáo có bố cục cố định	PDF

Kiểm tra kiến thức cuối bài

Câu 1. Kỹ thuật nào phù hợp khi tệp CSV lớn hơn RAM?

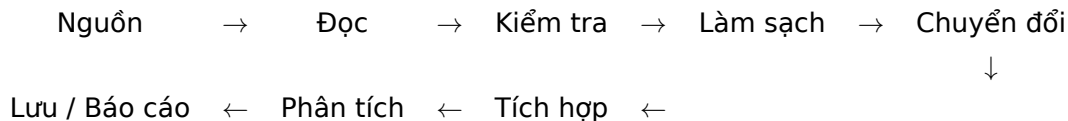
- A. Đọc tệp theo từng khối
- B. Chuyển mọi giá trị thành chuỗi
- C. Tạo bản sao thứ hai của tệp
- D. Chỉ dùng head ()

Câu 2. Định dạng nào đặc biệt phổ biến khi trao đổi dữ liệu qua Web API?

- A. JSON
- B. PDF
- C. XLS
- D. SQLite

Tổng kết

Quy trình tổng quát khi làm việc với dữ liệu đa nguồn:



Mục tiêu là chuyển dữ liệu thô, phân tán thành dữ liệu phân tích đáng tin cậy.

Các công cụ chính

Nguồn dữ liệu	Công cụ Python
CSV	<code>pd.read_csv(), to_csv()</code>
Tập văn bản số với NumPy	<code>loadtxt(), genfromtxt(), savetxt()</code>
Excel	<code>read_excel(), ExcelWriter()</code>
JSON	<code>read_json(), json_normalize()</code>
HTML	<code>read_html()</code>
PDF	<code>pdfplumber</code>
SQLite	<code>sqlite3, read_sql_query(), to_sql()</code>
MongoDB	<code>pymongo, json_normalize()</code>

Thông điệp kết thúc

Truy xuất, chuẩn bị và lưu trữ dữ liệu không phải là các bước kỹ thuật phụ. Chúng quyết định liệu các phân tích, trực quan hóa và mô hình Machine Learning sau đó có đáng tin cậy hay không.

Pipeline dữ liệu đúng $\Rightarrow$ kết quả phân tích đáng tin cậy